

Aperture — Low-Level Design

Module: Reviews

Version: v1.0

Module Owner: Community Platform

Dependencies:

- Authentication Module
 - Users Module
 - Metadata Module
 - Notification Module
 - Discovery Module
 - Moderation Module (future)
-

1. Overview

The Reviews module manages all user-generated opinions associated with movies and TV shows.

Unlike ratings, which are numerical signals primarily used for recommendation and aggregation, reviews are long-form pieces of content that form the foundation of Aperture's community experience.

Reviews should therefore be modeled as independent entities with their own lifecycle rather than being embedded inside movie records.

This enables:

- richer discussion
 - social engagement
 - moderation
 - discoverability
 - recommendation of reviews themselves
-

2. Design Goals

The Reviews module should:

- Encourage thoughtful discussion

- Support both short and long-form reviews
 - Scale independently from metadata
 - Enable social interactions
 - Integrate naturally with recommendation systems
 - Provide moderation hooks
 - Maintain data integrity
-

3. Module Ownership

Owns

- Reviews
 - Ratings
 - Review edits
 - Review deletion
 - Likes
 - Spoiler flags
 - Review visibility
-

Reads

- User profiles
 - Movies
 - TV Shows
 - Follow relationships
-

Never Modifies

- Metadata
 - User profiles
 - Lists
 - Recommendations
-

4. Responsibilities

The Reviews module owns:

Reviews

- Create

- Read
 - Update
 - Delete
-

Ratings

- Star ratings
 - Rating updates
 - Rating history (future)
-

Engagement

- Likes
 - Comments (future)
 - Shares (future)
 - Helpful votes (future)
-

Visibility

- Public
 - Private
 - Followers only (future)
-

Moderation

- Report review
 - Hide review
 - Review state
 - Soft deletion
-

5. High-Level Workflow

Write Review



Validate Request



Verify Movie Exists



Verify User



Create Rating



Create Review



Publish Activity



Update User Statistics



Trigger Recommendation Refresh



Return Review

Edit Review



Verify Ownership



Validate Changes



Persist Update

↓

Refresh Activity

↓

Return Updated Review

Delete Review

↓

Verify Ownership

↓

Soft Delete

↓

Update Statistics

↓

Refresh Related Data

↓

Return Success

6. Review Lifecycle

Every review moves through defined states.

Draft (future)

↓

Published

↓

Edited



Reported (optional)



Hidden (optional)



Deleted

Soft deletion is preferred to preserve moderation history.

7. Data Model

Primary entities:

Review

Stores:

- author
 - content
 - title (optional)
 - rating
 - spoiler flag
 - visibility
 - timestamps
-

Rating

Stores:

- user
- title
- numerical score
- creation date
- update date

Ratings remain separate so they can exist even without written reviews.

Review Like

Stores:

- review
- user
- timestamp

Future:

- Comments
 - Helpful votes
 - Review bookmarks
-

8. Business Rules

A user may have:

- One rating per title
- One review per title (initially)

Future support for review revisions rather than multiple concurrent reviews.

Review updates should preserve edit history where practical.

9. Validation Rules

Review:

- Minimum length
- Maximum length
- Supported formatting
- Spoiler declaration

Rating:

- Valid rating scale
- Existing title
- Authenticated user

Engagement:

- Prevent duplicate likes
 - Prevent self-like abuse (configurable)
-

10. Database Interactions

Reads:

- Reviews by title
- Reviews by user
- Review detail
- Like counts

Writes:

- Review creation
- Rating updates
- Like creation
- Moderation state

Indexes should optimize:

- title lookups
 - author lookups
 - creation date
 - popularity
-

11. Activity Integration

Review creation should publish an activity event.

Examples:

- User reviewed a movie
- User updated a rating
- User liked a review

This activity becomes available to:

- followers
- profile pages
- future recommendation systems

The Reviews module should not generate feeds directly; it only emits events that other modules consume.

12. Recommendation Integration

Reviews provide valuable preference signals.

Examples:

- highly rated genres
- preferred directors
- recurring themes
- favorite actors

Recommendation systems should consume review signals asynchronously rather than during the review creation request.

13. Moderation Strategy

The module should expose moderation hooks.

Future capabilities include:

- reporting
- spam detection
- offensive content review
- duplicate review detection
- AI-assisted moderation

Moderation logic should remain independent from review creation.

14. Caching Strategy

Cache candidates:

- popular reviews
- top reviews
- review summaries
- engagement counts

Avoid caching newly created reviews for long durations.

Popularity changes frequently.

15. Performance Considerations

Potential hotspots:

- blockbuster releases
- highly active reviewers
- viral reviews

Mitigation strategies:

- pagination
- caching
- asynchronous statistic updates
- lazy loading comments
- optimized indexes

Review retrieval should remain efficient even for titles with thousands of reviews.

16. Security Considerations

Review ownership must be verified for:

- edits
- deletion

Rich text should be sanitized.

Rate limiting should prevent:

- spam
- automated posting
- abuse

Future:

- content fingerprinting
 - abuse detection
-

17. Error Handling

Representative failures:

- title not found
- duplicate review
- invalid rating
- unauthorized edit
- review too long
- moderation restriction

Errors should be specific enough for clients while avoiding unnecessary internal detail.

18. Testing Strategy

Unit Tests

- rating validation
 - spoiler logic
 - business rules
-

Integration Tests

- review creation
 - updates
 - deletion
 - engagement
-

API Tests

- CRUD operations
 - authorization
 - pagination
 - sorting
-

Performance Tests

- high-volume review retrieval
- engagement updates
- popular title scenarios

19. Future Service Extraction

The Reviews module is a strong candidate for future extraction.

Reasons:

- high write volume
- independent scaling requirements
- social functionality
- moderation pipeline

Potential published events:

- ReviewCreated
- ReviewUpdated
- ReviewDeleted
- RatingChanged

Potential consumers:

- Feed Generation
- Notifications
- Recommendation Engine
- Analytics

Because the module already communicates through domain events conceptually, migration to a dedicated service would require minimal changes to consuming modules.

20. Future Evolution

Potential enhancements:

- Rich text formatting
- Embedded media
- Review comments
- Quote highlights
- AI-generated summaries
- Sentiment analysis
- Review collections
- Featured reviews
- Editorial picks

The architecture should support these additions without breaking existing APIs.

21. Interview Discussion Points

Be prepared to explain:

- Why separate Ratings from Reviews?
 - Why model Reviews as independent content instead of child objects?
 - How would you prevent spam and review bombing?
 - Why use soft deletion instead of hard deletion?
 - How would you scale review retrieval for blockbuster releases?
 - What events would the Reviews module publish in a future event-driven architecture?
-

22. Summary

The Reviews module establishes Aperture as a community platform rather than simply a content catalog.

By treating reviews as first-class entities with independent lifecycles, social interactions, and moderation capabilities, the module provides a scalable foundation for future feeds, AI insights, and recommendation systems while remaining cleanly separated from metadata and user management.